

Level 1 Canvas – System Context for Global Digital Bank (GDB)

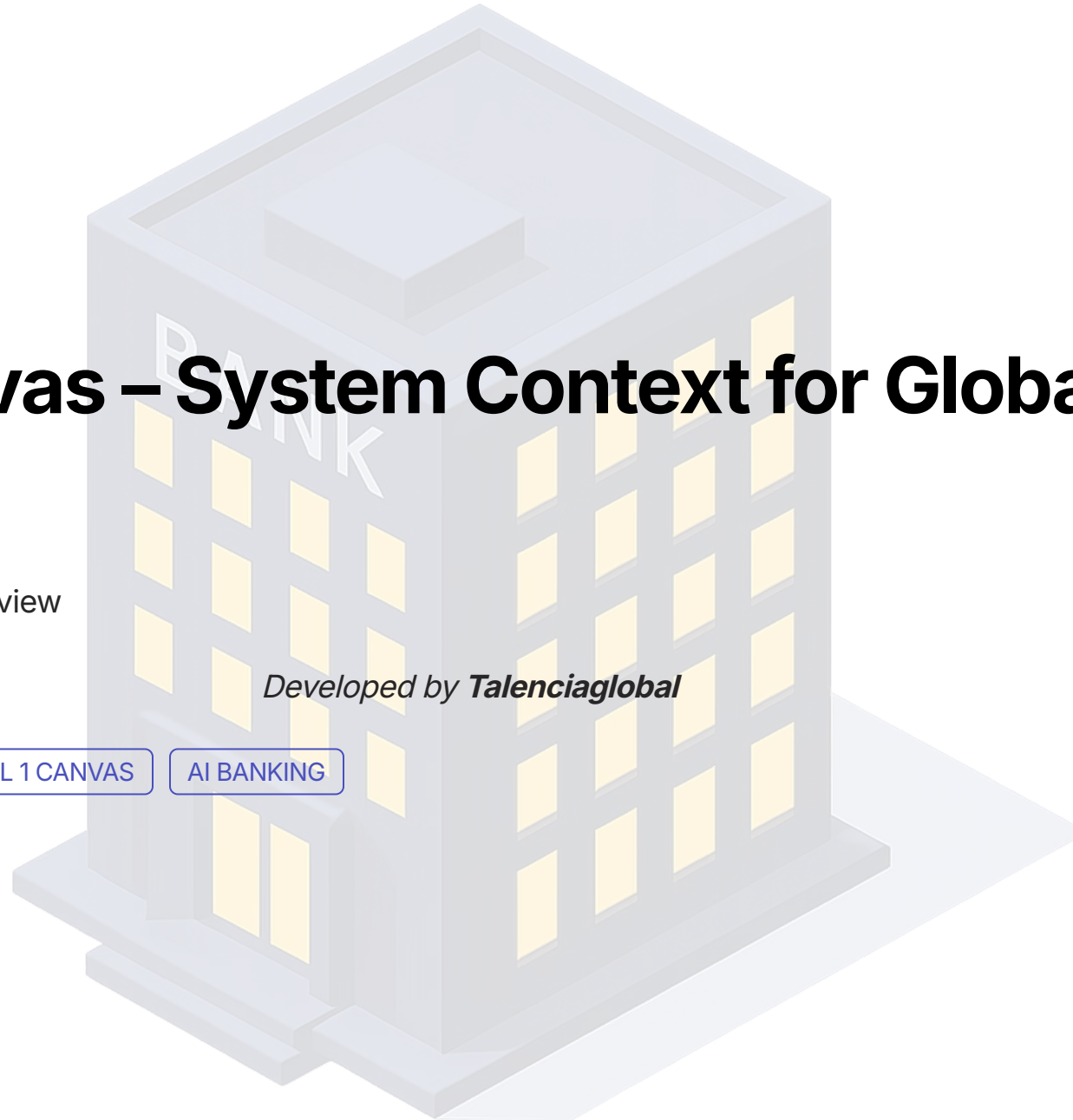
The 1,000,000 (1 Million feet) view

Developed by Talenciaglobal

SYSTEM ARCHITECTURE

LEVEL 1 CANVAS

AI BANKING





Level 1 Canvas - Quick Reference

This Level 1 Canvas now serves as the comprehensive system context for the Global Digital Bank project. It provides all the necessary high-level understanding before moving to Level 2 (Solution Architecture), where specific technologies and platform decisions will be made.

Component	Key Insights
Purpose	Secure, intelligent CLI banking with 30+ features, including 4 agentic AI capabilities
Users	Customers (primary), Admins, Compliance Officers, Support Staff
Scale	10K→100K accounts, 500→50K MAU, 100→10K RPS over 3 years
Criticality	1-day outage → ₹5M+ loss, severe reputational damage
Compliance	GDPR, PCI-DSS, RBI, SOC2, SOX
Key Flows	6 critical user journeys: Onboarding, Transaction, AI, Security, Admin, Batch
Components	8 subsystems including AI/Agent Layer, File Manager, Security Manager
Dependencies	File System, LLM Provider, Time Service
Anchors	Chase, PayPal, Revolut (similar operations, different scale/interface)
Assumptions	File performance, LLM availability, PIN security, single-user concurrency
Justification	Complexity, NFRs, AI integration, knowledge transfer, future proofing



Industry fact: According to McKinsey, digital banking platforms that document architecture upfront reduce technical debt by up to 40% and cut time-to-feature by 30%.

Purpose Statement

1-sentence mission:

Global Digital Bank (GDB) delivers a secure, intelligent, and comprehensive digital banking platform that empowers customers with seamless account management, real-time transactions, and AI-driven financial insights through an accessible command-line interface.

Why now:

Digital Transformation

Traditional banks are losing customers to fintech competitors offering superior digital experiences

AI Revolution

Agentic AI capabilities (LangChain/LangGraph) can now provide personalized financial advice and fraud detection at scale

Skill Development

There's a critical need for developers who can architect complex systems combining traditional banking with modern AI

Market Gap

No existing open-source banking platform demonstrates integration of traditional operations with agentic AI in an educational, modular format

 Industry anecdote: JPMorgan Chase invested \$12B in technology in 2023 alone — more than many fintech startups' total valuations — underscoring the urgency of digital transformation in banking.



External Ecosystem – Users (Human)

User Type	Role	Key Needs
Primary: Bank Customers	Individual account holders	Manage accounts, transact securely, get financial advice
Secondary: Bank Administrators	Bank employees with admin privileges	Oversee operations, generate reports, manage security
Tertiary: Compliance Officers	Regulatory staff	Audit trails, regulatory reporting, suspicious activity monitoring
Tertiary: Customer Support	Bank support staff	Access account info, assist with issues, view transaction history

Actors (Machine):

Actor	Purpose	Interaction Pattern
LangChain/LangGraph Agent	AI-powered features	Internal tool calls, natural language processing
LLM Provider (OpenAI/Anthropic)	Natural language understanding	API calls for NLQI, financial advisor, predictive analytics
File System	Data persistence	Read/write CSV and log files
Backup Service	Disaster recovery	Scheduled backups of accounts.csv
Audit Logger	Compliance	Append-only transaction logging
Notification Service	User alerts	Email/SMS for suspicious activity (future)

Integrations:

Integration	Type	Mandatory/Optional
OpenAI GPT API	AI/NLP	Optional (can use open-source model)
File System	Storage	Mandatory
SMTP Email Server	Notifications	Optional (for future)
SMS Gateway	Notifications	Optional (for future)
Kafka/RabbitMQ	Message Queue	Optional (for future scaling)



Scale & Complexity Estimates

User Base:

Metric	Current (Phase 1)	Expected (1 Year)	Expected (3 Years)
Total Accounts	1,000	10,000	100,000
Monthly Active Users	500	5,000	50,000
Concurrent Users	10	100	1,000
Admin Users	2	10	50

Data Volume:

Metric	Current (Phase 1)	Expected (1 Year)	Expected (3 Years)
Storage Required	50 MB	500 MB	5 GB
Daily Data Ingestion	500 KB	5 MB	50 MB
Transaction Records	1,000/day	10,000/day	100,000/day
Retention Period	1 year	5 years	7 years (regulatory)

Transaction Volume:

Metric	Current (Phase 1)	Expected (1 Year)	Expected (3 Years)
Peak RPS (Reads)	100	1,000	10,000
Peak RPS (Writes)	50	500	5,000
Average RPS	10	100	1,000
Burst Patterns	Salary days (month-end)	Salary days + Festival seasons	Multiple peaks



Industry fact: UPI in India processed over 13 billion transactions in a single month in 2024 — demonstrating the explosive scale digital banking infrastructure must be designed to handle.



Geographic Distribution

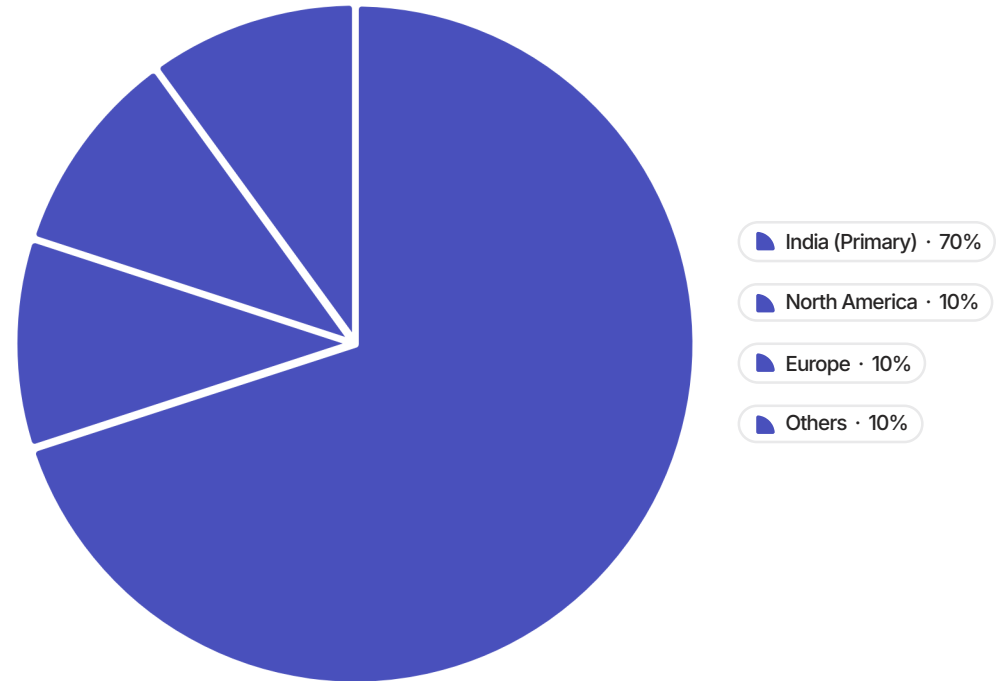
Region Distribution

India (Primary): 70% — <100 ms

North America: 10% — <300 ms

Europe: 10% — <250 ms

Others: 10% — <400 ms



Industry anecdote: Revolut serves 40+ million customers across 35+ countries, with latency SLAs varying dramatically by region — a key lesson for any globally distributed banking platform.

Criticality & Impact

Business Criticality:

Outage Duration	Impact	Financial Loss	Reputational Damage
1 Minute	Minor inconvenience	₹0 - ₹1,000	Negligible
1 Hour	Transaction delays, user frustration	₹10,000 - ₹50,000	Moderate
1 Day	Significant business disruption, regulatory implications	₹500,000 - ₹5,000,000	Severe
1 Week	Potential business failure, legal action	₹50,000,000+	Catastrophic

 Industry fact: The 2021 TSB Bank outage in the UK cost the bank over £330 million and resulted in 80,000 customers leaving — a stark reminder of the catastrophic cost of downtime in banking.



Compliance & Security Posture

Compliance/Regulatory Constraints:

Regulation	Applicability	Impact
GDPR	European users	Data privacy, right to deletion, breach notification
PCI-DSS	Payment data	Encryption, access control, audit trails
RBI Guidelines	Indian operations	KYC, transaction monitoring, data localisation
SOC2	Customer trust	Security, availability, processing integrity, confidentiality, privacy
SOX	Financial reporting	Audit trails, data integrity

Security Posture:

Aspect	Sensitivity Level	Protection Required
PII (Name, Age)	High	Encryption at rest & in transit
Financial Data (Balance)	Critical	Encryption, access control
Transaction History	Critical	Audit logging, tamper-proof storage
PIN/Passwords	Critical	Hashed storage, never exposed
Admin Credentials	Critical	MFA, strict access control
Log Files	Medium	Anonymized where possible



Industry fact: The average cost of a financial data breach reached \$5.9 million in 2023 (IBM Cost of a Data Breach Report) — nearly double the cross-industry average, making security posture a non-negotiable investment.



User Journey (Before & After)

5 Seconds Before:

User	Scenario	What User Is Doing	Context
Customer	Opening banking app/terminal	"I need to check my balance" or "I want to transfer money"	
Admin	Logging into admin portal	"I need to generate a report for today's transactions"	
Support Agent	Accessing account information	"A customer called about a missing transaction"	
Compliance Officer	Accessing audit logs	"We need to produce records for a regulatory review"	

5 Seconds After:

User	Scenario	What User Does Next	System Delivers
Customer	Reads balance, or confirms transfer, or follows AI advice		Transaction confirmation, balance display, AI recommendation
Admin	Reviews report, takes action on flagged accounts		Generated report, summary dashboard
Support Agent	Answers customer query, resolves issue		Complete transaction history, account details
Compliance Officer	Reviews logs, investigates suspicious patterns		Filtered audit reports, flagged transactions



Key User Flows (Critical Journeys)

01

Create Account → Deposit → Balance Inquiry

Onboarding flow

02

Withdraw → Transfer Funds → Transaction History

Transaction flow

03

AI Natural Language Query → Action → Confirmation

AI-assisted flow

04

Fraud Detection → Block → Verify → Proceed

Security flow

05

Admin Login → Report Generation → Export

Admin flow

06

Import Accounts → Bulk Create → Verification

Batch processing flow



Industry anecdote: Monzo mapped over 200 distinct user journeys before launching their banking app — a practice that directly contributed to their 99.9%+ uptime and industry-leading NPS scores.

High-Level Components (Black Box View)

Major Subsystems:

Account Management

CRUD operations for accounts

Create, search, list, close, reopen, rename

Transaction Engine

Process deposits, withdrawals, transfers

Balance management, rule enforcement

Analytics Engine

Generate insights

Top balances, averages, interest calculation

AI/Agent Layer

Intelligent features

Fraud detection, financial advisor, NLQI, predictive dashboard

File Manager

Data persistence

Read/write CSV, transaction logs

Security Manager

Authentication & authorization

PIN verification, role-based access

Admin Console

Administrative functions

Report generation, bulk operations

Notification Service

User alerts

Logging, future email/SMS

External Dependencies & Entry Points

External Dependencies:

Dependency	Type	Purpose	Failure Mitigation
File System	Infrastructure	Data persistence	Fallback to in-memory, retry logic
LLM Provider API	Service	NLQI, advisory	Local fallback model, graceful degradation
Time Service	System	Timestamps, date tracking	System time sync, fallback

Entry Points:

Entry Point	Client	Protocol	Authentication
CLI Menu	Human users	Local terminal	PIN (for sensitive ops)
Admin CLI	Admin users	Local terminal	Admin credentials + PIN
Natural Language Interface	Human users	Local terminal	Account-level, context-aware
AI Agent Tools	LangChain/LangGraph	Function calls	Internal (system-level)
File Import/Export	Admin users	Local files	Admin-level access



Comparison Anchors

Similar Systems (Real-World Examples):

System	Similarities	Differences
Chase Mobile Banking App	Account management, transfers, transaction history	Closed-source, mobile-first, massive scale
PayPal	Fund transfers, transaction history, security	Payment gateway focus, web-first
Revolut	Modern banking, AI features, multi-currency	Production-scale, mobile-first, regulated
Simple (BBVA)	User-friendly banking, financial insights	Decommissioned, web/mobile only
Kubernetes + Microservices Example	Modular architecture, scaling patterns	Not banking-specific, different domain
LangChain Agents (AI Examples)	Agentic AI patterns, tool calling	Not banking-specific, demo scale

What is Similar

- Core banking operations (CRUD, transactions, balances)
- User authentication and authorization patterns
- Data persistence and audit logging requirements
- Modular architecture with separation of concerns
- AI integration for intelligent features

What is Different

- CLI-only interface (vs. web/mobile-first)
- Educational scope (vs. production financial system)
- Single developer (vs. large engineering teams)
- File-based persistence (vs. enterprise databases)
- Agentic AI emphasis (vs. basic ML models)
- Open-source, customizable (vs. proprietary)

? Key Unanswered Questions (Assumptions to Validate)

Biggest Unknowns:

- 1

Scale Validation
Can file-based persistence handle 10,000+ accounts with acceptable performance?
- 2

LLM Dependency
Will the LLM provider API be available, or should we build a fallback mechanism?
- 3

Concurrency Handling
How to handle concurrent operations on the same account?
- 4

Error Recovery
What happens if accounts.csv gets corrupted?
- 5

Data Migration
How to handle schema changes in future versions?
- 6

Admin Access
Should there be a separate admin login mechanism?

Break-If-Wrong Assumptions:

Assumption	Impact If Wrong	Mitigation
"File I/O is fast enough for all operations"	Performance degradation at scale	Introduce in-memory cache, batch writes
"The system will only have one user at a time"	Race conditions, data corruption	Implement file locking, ACID transactions
"PIN is sufficient for security"	Security breaches	Add MFA, hashed storage
"LLM provider will always be available"	AI features fail	Fallback to rule-based systems
"Accounts.csv format will remain stable"	Data migration issues	Versioned file format, migration scripts

What to Learn Before Step 2:

- Performance benchmarks for 10,000 accounts with CSV file operations
- LLM API costs and rate limits
- File locking mechanisms for Python
- Data migration best practices
- Security best practices for PIN storage



Justification for Architecture Effort

Why Explicit Architecture is Required:

Complexity 30+ features spanning multiple domains (transactions, analytics, AI)	NFR Criticality Performance, security, and compliance are non-negotiable
Future Extensibility Need to add features without breaking existing functionality	AI Integration Agentic AI layer requires careful design to integrate with traditional banking
Knowledge Transfer Architecture documents serve as a learning resource	Evolution Path Need to plan for future scaling (database, web, mobile)

Risk of Not Doing Architecture Upfront:

Risk	Impact	Likelihood
Technical Debt	Code becomes unmaintainable, features take longer to add	High
Performance Bottlenecks	System fails at scale, users experience latency	High
Security Vulnerabilities	Data breaches, regulatory fines	Medium
Integration Failures	AI features don't integrate properly with banking operations	Medium
Rediscovery of Solutions	Implementing patterns that should have been designed upfront	Very High
Scope Creep	Features added without understanding system impact	High



Industry anecdote: According to the Standish Group CHAOS Report, only 31% of software projects succeed without upfront architecture. Projects with documented architecture are 2.5x more likely to be delivered on time and on budget.



Deliverable Summary (Checklist)

- ✓ 1-page system description written – Purpose, scope, user types
- ✓ Justification for architecture included – Why explicit architecture is required
- ✓ Scale, complexity, and criticality stated – Quantitative estimates for users, data, transactions
- ✓ Comparison anchors documented – Real-world systems with similarities and differences
- ✓ High-level components and external integrations listed – 8 subsystems, 3 external dependencies
- ✓ User journey described (before/after) – 4 user scenarios, 6 key flows
- ✓ Assumptions explicitly called out – 5 break-if-wrong assumptions, 4 learning priorities
- ✓ External ecosystem mapped – 4 user types, 7 machine actors, 5 integrations
- ✓ Security posture defined – Classification of sensitive data, protection levels
- ✓ Regulatory constraints identified – 5 key regulations with compliance impact

*Developed by **Talenciaglobal** — advancing architecture education for the next generation of banking engineers.*